EXPERIMENT 6: Gradient Descent, SGD, Momentum and Adam Comparison

1. Aim

To implement Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Momentum from scratch using NumPy, visualize their optimization paths on a two-dimensional loss contour, and compare SGD, SGD with Momentum, and Adam using a Keras regression model.

2. Mathematical Formulae

Batch Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Stochastic Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla J_i(\theta_t)$$

Momentum

$$v_t = \beta v_{t-1} + \nabla J(\theta_t)$$

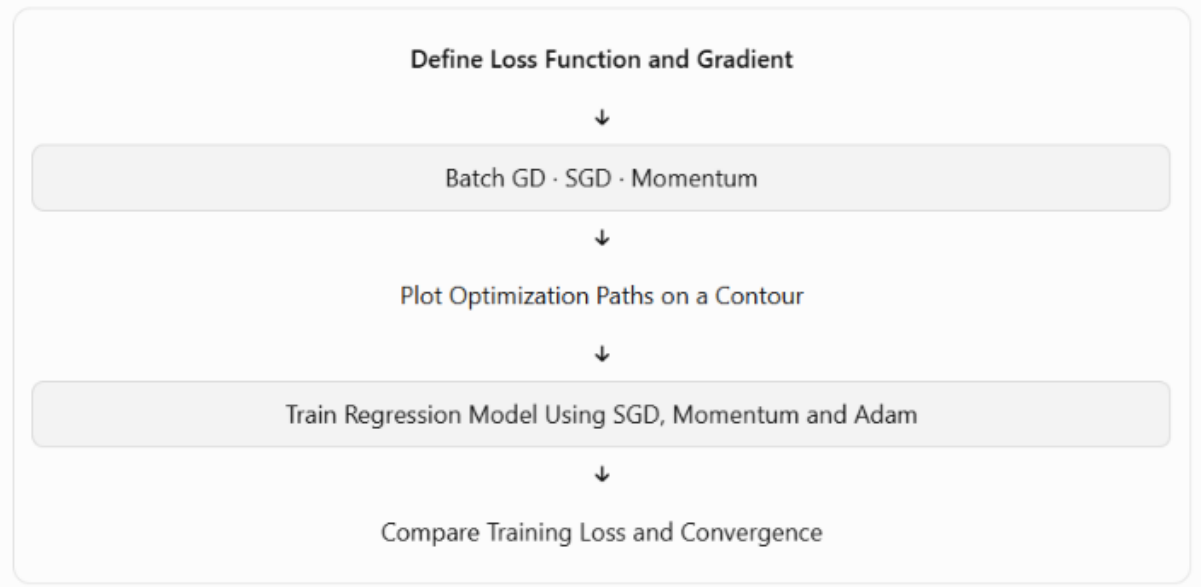
$$\theta_{t+1} = \theta_t - \eta v_t$$

Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Where θ represents model parameters, η is the learning rate, and β is the momentum coefficient.

3. System Architecture



4. Algorithm / Procedure

1. Import NumPy, Matplotlib and TensorFlow/Keras.
2. Define a quadratic loss function and its gradient.
3. Implement Batch Gradient Descent, SGD and Momentum.
4. Plot the optimization paths on a two-dimensional contour plot.
5. Generate a synthetic delivery-time dataset with distance, traffic, weight and time of day as inputs.
6. Train identical regression models using SGD, SGD with Momentum and Adam.
7. Plot and compare the training losses.

5. Python Program

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras

np.random.seed(42)
tf.random.set_seed(42)

# Part A: Define the loss function
def loss(p):
```

```
    x, y = p
    return (x - 3)**2 + 4 * (y + 1)**2

def grad(p):
    x, y = p
    return np.array([2 * (x - 3), 8 * (y + 1)])

start = np.array([-4.0, 4.0])
steps = 80

# Batch Gradient Descent
p = start.copy()
gd = [p.copy()]

for _ in range(steps):
    p -= 0.08 * grad(p)
    gd.append(p.copy())

# Stochastic Gradient Descent
p = start.copy()
sgd = [p.copy()]

for _ in range(steps):
    noisy_gradient = grad(p) + np.random.normal(0, 1.2, 2)
    p -= 0.025 * noisy_gradient
    sgd.append(p.copy())

# Momentum
p = start.copy()
v = np.zeros(2)
momentum = [p.copy()]

for _ in range(steps):
    v = 0.85 * v + grad(p)
```

```
    p -= 0.035 * v
    momentum.append(p.copy())

# Plot contour and optimization paths
x = np.linspace(-5, 6, 150)
y = np.linspace(-4, 6, 150)
X, Y = np.meshgrid(x, y)
Z = (X - 3)**2 + 4 * (Y + 1)**2

plt.figure(figsize=(9, 6))
plt.contour(X, Y, Z, levels=25)
plt.plot(*np.array(gd).T, marker=".", label="Batch GD")
plt.plot(*np.array(sgd).T, marker=".", label="SGD")
plt.plot(*np.array(momentum).T, marker=".", label="Momentum")
plt.xlabel("Parameter x")
plt.ylabel("Parameter y")
plt.title("Optimization Paths")
plt.legend()
plt.show()

# Part B: Synthetic delivery-time regression dataset
n = 1000

distance = np.random.uniform(1, 30, n)
traffic = np.random.uniform(1, 10, n)
weight = np.random.uniform(0.5, 20, n)
time_of_day = np.random.uniform(0, 24, n)

X_data = np.column_stack([
    distance, traffic, weight, time_of_day
])

delivery_time = (
    5 + 1.8 * distance + 2.2 * traffic
```

```
    + 0.3 * weight
    + 0.5 * np.sin(time_of_day / 24 * 2 * np.pi)
    + np.random.normal(0, 3, n)
)

# Standardise inputs and target
X_mean = X_data.mean(axis=0)
X_std = X_data.std(axis=0)
X_data = (X_data - X_mean) / X_std

y_mean = delivery_time.mean()
y_std = delivery_time.std()
y_data = (delivery_time - y_mean) / y_std

# Train identical models with different optimisers
optimizers = {
    "SGD": keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ),
    "Adam": keras.optimizers.Adam(learning_rate=0.01)
}

histories = {}

for name, optimizer in optimizers.items():
    model = keras.Sequential([
        keras.layers.Input(shape=(4,)),
        keras.layers.Dense(16, activation="relu"),
        keras.layers.Dense(1)
    ])

    model.compile(optimizer=optimizer, loss="mse")
```

```
    history = model.fit(
        X_data, y_data,
        epochs=50,
        batch_size=32,
        validation_split=0.2,
        verbose=0
    )

    histories[name] = history.history

# Compare training and validation losses
plt.figure(figsize=(9, 5))

for name, history in histories.items():
    plt.plot(history["loss"], label=name)

plt.xlabel("Epoch")
plt.ylabel("Training MSE")
plt.title("Optimiser Comparison")
plt.legend()
plt.show()

plt.figure(figsize=(9, 5))

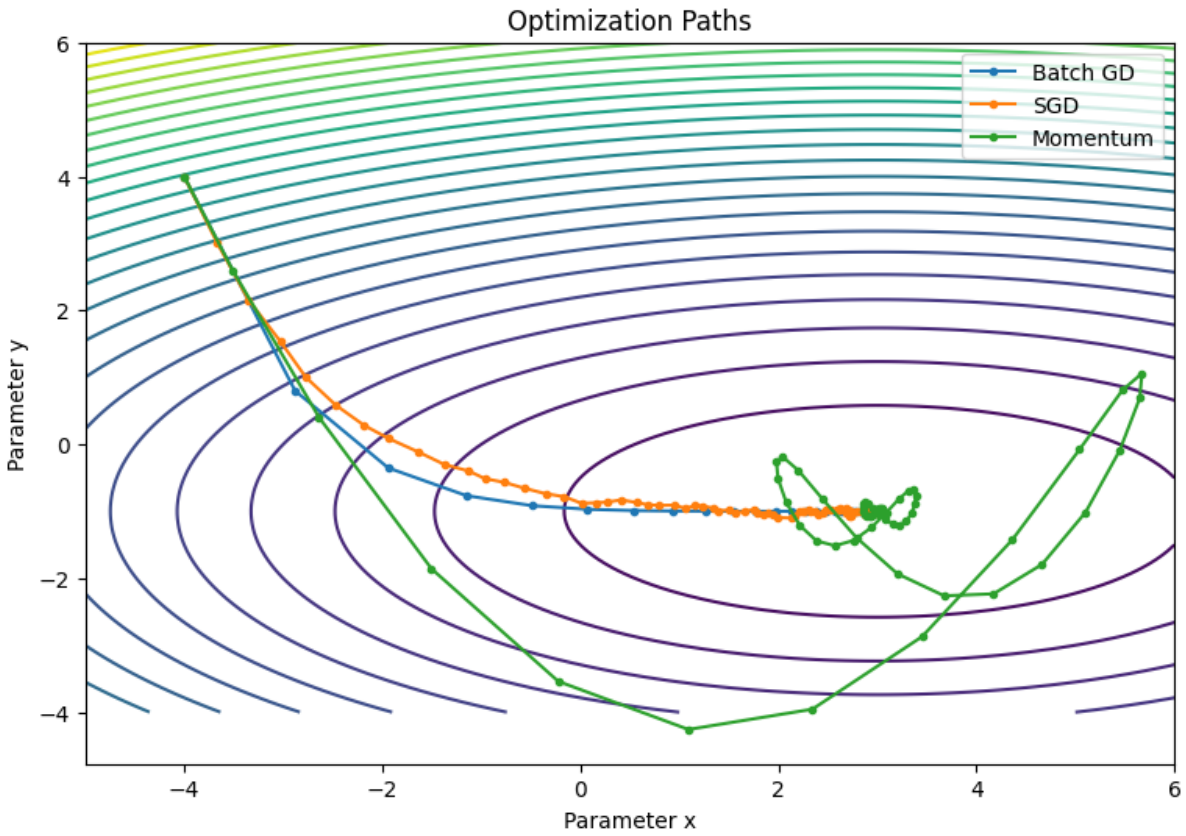
for name, history in histories.items():
    plt.plot(history["val_loss"], label=name)

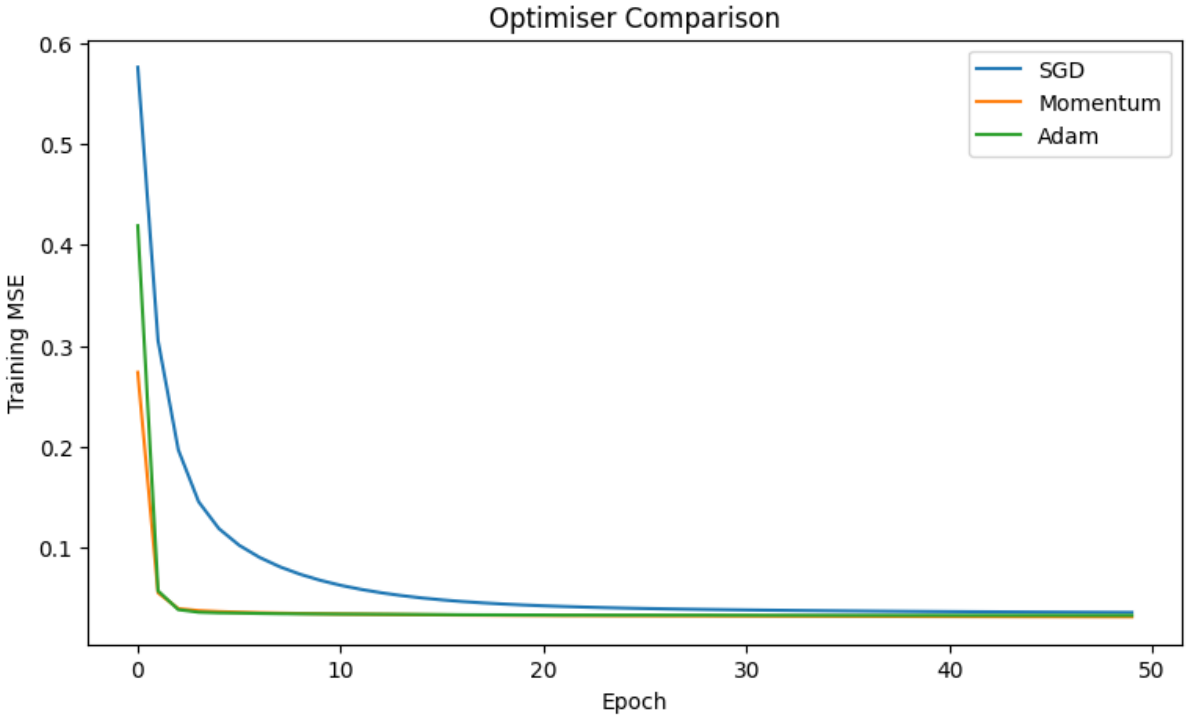
plt.xlabel("Epoch")
plt.ylabel("Validation MSE")
plt.title("Validation Loss Comparison")
plt.legend()
plt.show()

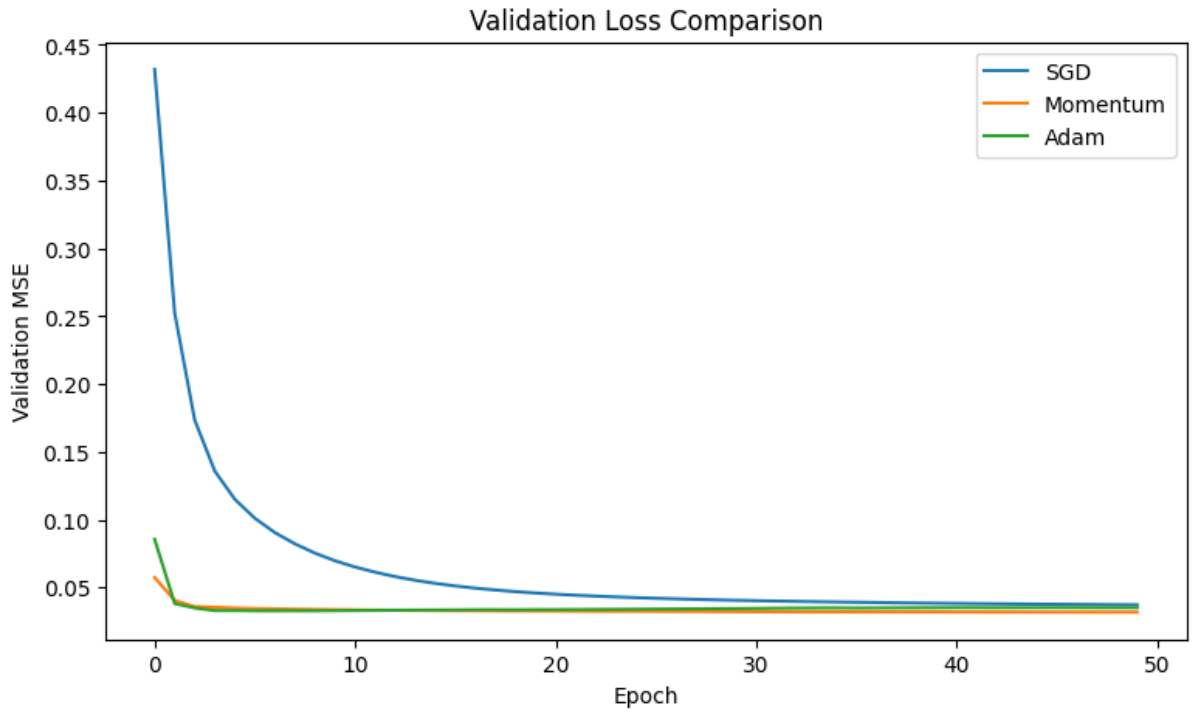
for name, history in histories.items():
```

```
    print(
        name,
        "Final training loss:", round(history["loss"][-1], 4),
        "Final validation loss:", round(history["val_loss"][-1], 4)
    )
```

6. Expected Output


Optimization Paths
Batch GD
SGD
Momentum
Parameter y
Parameter x
6
4
2
0
−2
−4
−4
−2
0
2
4
6


Optimiser Comparison
SGD
Momentum
Adam
Training MSE
Epoch
0.6
0.5
0.4
0.3
0.2
0.1
0
10
20
30
40
50




```
SGD Final training loss: 0.0361 Final validation loss: 0.0373
Momentum Final training loss: 0.0317 Final validation loss: 0.0319
Adam Final training loss: 0.0331 Final validation loss: 0.0352
```

7. Result

Batch Gradient Descent, SGD and Momentum were implemented using NumPy. Their optimization paths were visualized, and regression models trained using SGD, Momentum and Adam were compared using training and validation loss.